

Report Task 5 — Forma Canonica GERANIO e Transition Matrix MTM ↔ EvoMine

Tirocinio: "Profili evolutivi multilivello di reti complesse tramite temporal network motif"

Data: 2026-07-10 | Autore: Carlo Invernizzi

1. Obiettivo

Costruire una forma canonica condivisa tra MTM ed EvoMine per rendere le due transition matrix direttamente confrontabili sullo stesso spazio di codifica, usando lo schema di codifica GERANIO concordato con la correlatrice invece della codifica digit-code (encode_motif) impiegata nei Task 3 e 4.

2. Dataset e parametri

- Dataset: CollegeMsg, campione 5000 eventi (collegemsg_sample_5000.txt, stesso del Task 3/4)
- MTM: L_MAX=3, DELTA=3600s, consecutive=YES
- EvoMine: s=1 (support), e=3 (max_edges), T=event, $\delta=3600s$ (1h)

3. Step 1 — Forma canonica condivisa

Lo schema adottato (confermato dalla correlatrice) è semplice: i nodi vengono rinominati in ordine di prima apparizione (0, 1, 2, ...), gli archi vengono ordinati per (t_logico, src, dst) e la forma canonica risultante è la stringa "src_dst_t-src_dst_t-..." ottenuta concatenando gli archi così ordinati.

La scelta dei t_logici differisce per costruzione tra le due fonti: MTM usa un orologio sequenziale, in cui ogni arco della sequenza riceve una posizione temporale distinta e crescente (t=0, t=1, t=2, ...); EvoMine usa invece un orologio binario body/head, in cui tutti gli archi di preconditione ricevono t=0 e tutti gli archi di postcondizione ricevono t=1, indipendentemente da quanti archi compongono ciascun gruppo. Nel testo grezzo di EvoMine gli archi di head possono anche comparire prima di quelli di body nell'ordine di stampa: la pipeline raggruppa esplicitamente gli archi per etichetta (body prima, head dopo) prima della rinomina, così il risultato rispetta sempre t=0 prima di t=1 indipendentemente dall'ordine di stampa originale.

L'unione di tutte le stringhe canoniche prodotte dalla pipeline (pattern MTM completi, regole EvoMine complete, e i pezzi riga/colonna riga di preconditione e postcondizione derivati per le transition matrix) è stata salvata in canonical_mapping.csv, con codici corti p1, p2, ..., pN assegnati in ordine di peso combinato decrescente (somma dei conteggi MTM e dei supporti EvoMine in cui compare ciascuna stringa).

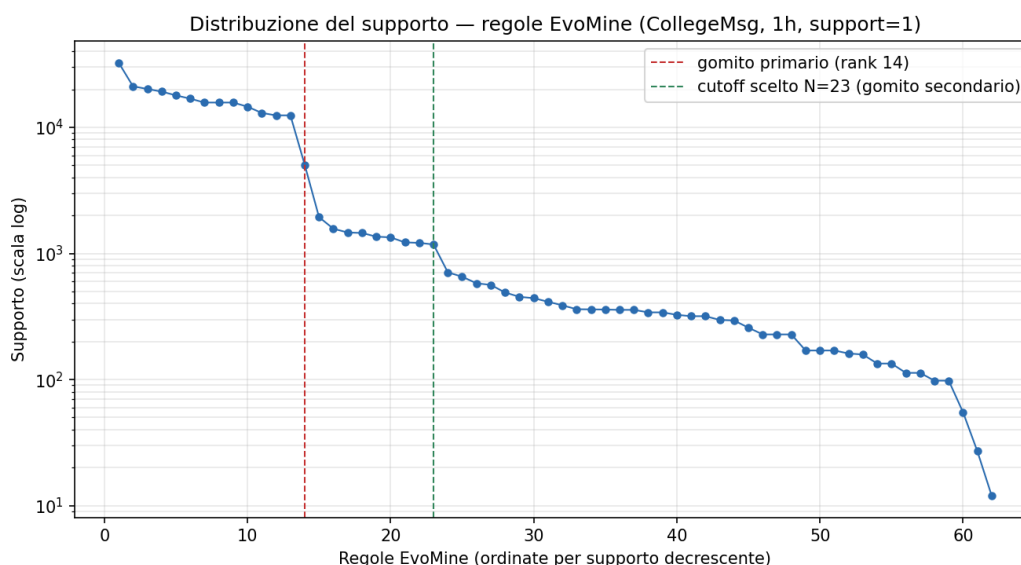
Totale codici	Da MTM	Da EvoMine	Condivisi (entrambi)
---------------	--------	------------	----------------------

148	69	76	3
-----	----	----	---

Solo 3 codici lunghi risultano condivisi tra MTM ed EvoMine — non un errore, ma conseguenza diretta dell'asimmetria di clock descritta sopra: le due convenzioni coincidono solo quando la preconditione EvoMine ha esattamente 1 arco body + 1 arco head, cioè quando la forma completa di una regola EvoMine a 2 archi combacia esattamente con la forma di un prefisso MTM a 2 archi ($t=0$, $t=1$ in entrambi i casi). Per questo i 3 codici in comune sono tutti forme complete di regole EvoMine a 2 archi, non colonne o righe isolate.

Codice lungo	Codice corto	Fonte
0_1_0-0_2_1	p18	entrambi
0_1_0-2_0_1	p21	entrambi
0_1_0-1_0_1	p36	entrambi

4. Step 2 — Distribuzione supporto EvoMine e selezione top N



Il gomito più marcato nella distribuzione (scala log) è al rank 14, dove il supporto scende da 5045 a 1941 (rapporto ≈ 2.60): separa nettamente le 14 regole a supporto altissimo — tutte pattern body-only, cioè senza postcondizione — dal resto della distribuzione. Tagliare esattamente a questo gomito produrrebbe però zero transizioni preconditione→postcondizione utilizzabili negli Step 3 e 4, poiché nessuna delle prime 14 regole ha una testa (head). Il secondo gomito visibile è al rank 23, dove il supporto scende da 1178 a 708 (rapporto ≈ 1.66): cade nell'intervallo 15–30 indicato come alternativa e include, oltre alle 14 regole body-only e alla regola head-only “ $\rightarrow 0_1_1$ ”, le prime 8 vere transizioni preconditione→postcondizione. Si è scelto quindi $N=23$ come cutoff, combinando un gomito secondario reale con la necessità di avere dati utilizzabili per gli step successivi.

Min	Mediana	P75	P90	P95	Max
12	401.0	1544.0	15714.0	19131.2	32571

Delle 23 regole selezionate, 14 sono pattern body-only (senza testa), 1 è head-only (“→0_1_1”) e 8 sono vere transizioni preconditione→postcondizione (body e head entrambi non vuoti) — queste ultime sono le uniche utilizzabili per costruire la transition matrix EvoMine dello Step 3.

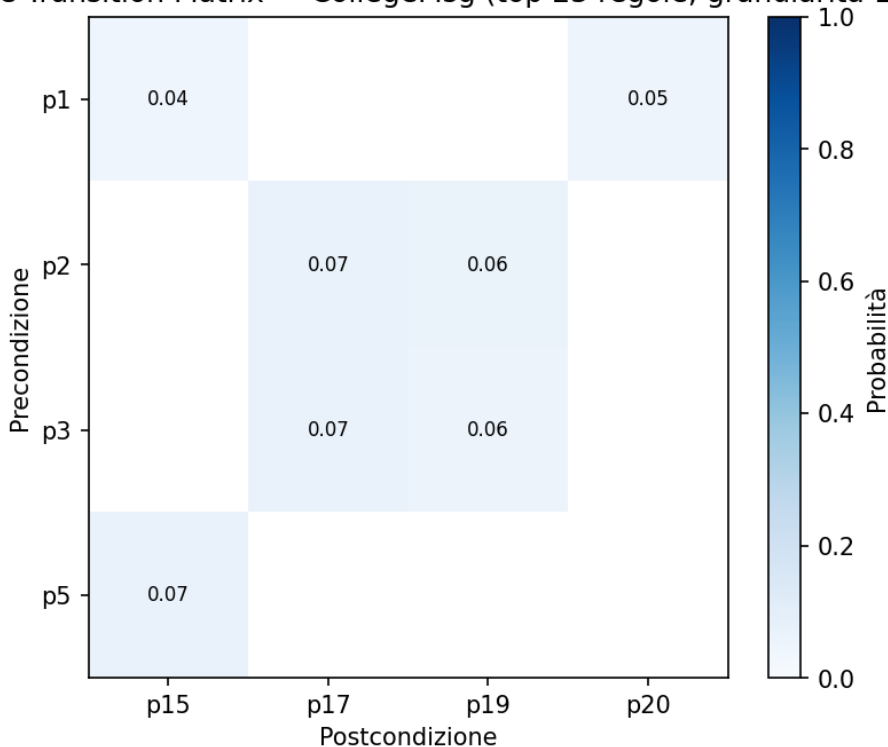
5. Step 3 — Transition matrix EvoMine

Per ciascuna delle 8 transizioni preconditione→postcondizione tra le top 23 regole si calcola la probabilità come:

$$P(pre \rightarrow post) = freq(pre \rightarrow post) / freq(pre)$$

dove $freq(pre)$ è il supporto della regola body-only con la stessa preconditione, cercata tra tutte le 62 regole minate (non solo le top 23), e $freq(pre \rightarrow post)$ è il supporto della regola completa. Il rapporto è sempre compreso tra 0 e 1 per l'antimonotonicità del supporto nel pattern mining: ogni estensione di un pattern (aggiungere archi alla preconditione per ottenere pre+post) non può avere supporto maggiore del pattern di partenza, quindi $freq(pre \rightarrow post) \leq freq(pre)$ sempre. In un caso (precondizione “0_1_0-2_3_0”, archi sconnessi) non esiste tra le 62 regole minate una regola body-only con questa stessa preconditione osservata come pattern chiuso a sé stante: la cella corrispondente resta NaN, senza ricomputare il conteggio dal dataset grezzo (fuori scopo qui).

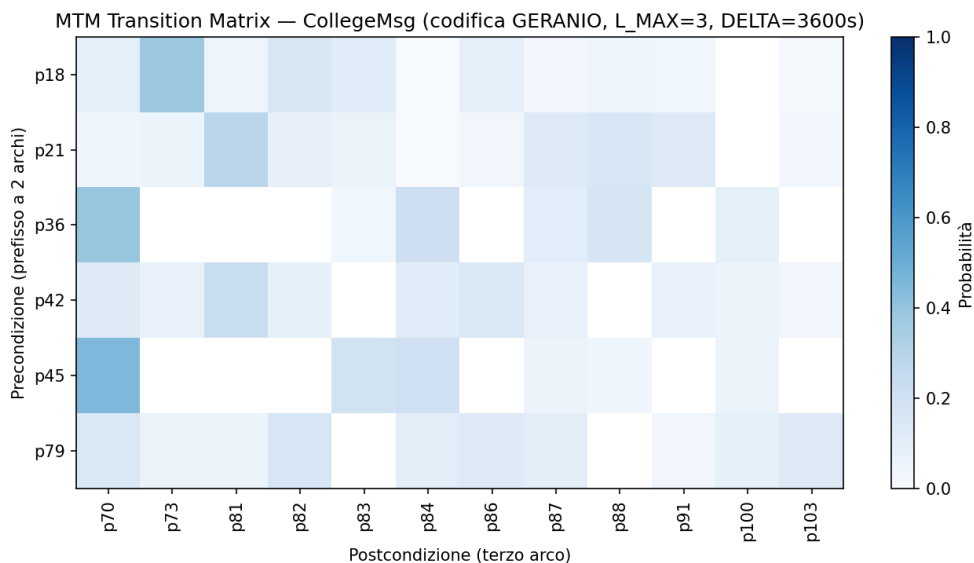
EvoMine Transition Matrix — CollegeMsg (top 23 regole, granularità 1h)



La matrice EvoMine risultante ha dimensioni 4×4 (precondizioni p1, p2, p3, p5 × postcondizioni p15, p17, p19, p20), con 8 celle strutturalmente popolate — 7 con probabilità calcolata e 1 lasciata a NaN per il motivo sopra descritto.

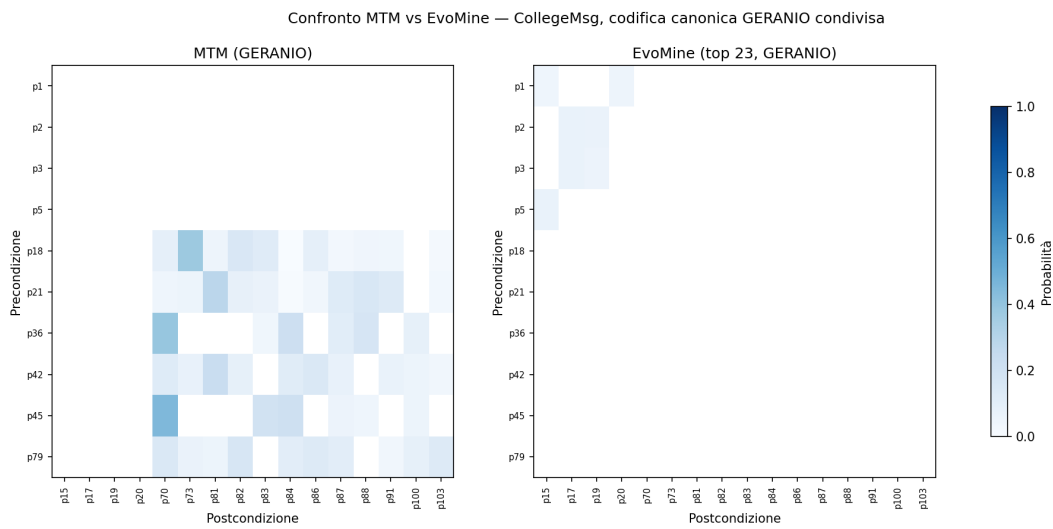
6. Step 4 — Transition matrix MTM con codifica GERANIO

È la stessa matrice del Task 3 (L_MAX=3, DELTA=3600s), con la stessa formula di probabilità — $P(\text{terzo evento} \mid \text{prefisso}) = \text{count}(\text{prefisso} \rightarrow \text{terzo evento}) / \text{count}(\text{prefisso} \rightarrow \text{qualsiasi evento})$, normalizzata sulla sola massa di prefissi che si sono effettivamente estesi, escludendo lo stato S di non-estensione, come corretto dalla correlatrice nel Task 3 — ma con righe e colonne rietichettate con i codici corti GERANIO (p1...p148) al posto della codifica digit-code encode_motif originale. Una verifica di continuità con il Task 3 conferma che il valore della transizione dominante 0102→03 resta 0.3752 (p18→p73) anche dopo la ricodifica.



La matrice MTM ha dimensioni 6×12 (6 prefissi a 2 archi × 12 possibili terzi archi), con 54 celle osservate su 72 possibili — identico al risultato del Task 3, cambia solo l'etichettatura degli assi.

7. Confronto finale



Celle in comune	Celle esclusive MTM	Celle esclusive EvoMine	Correlazione di Pearson
0	54	8	non calcolabile (0 celle in comune)

Le due matrici non condividono nessuna cella: la matrice MTM ha come colonne il terzo arco isolato (etichettato $t=2$ nell'orologio sequenziale), mentre le colonne EvoMine sono postcondizioni etichettate $t=1$ nell'orologio binario body/head — stringhe diverse anche a parità di struttura (src, dst). Con 0 celle in comune la correlazione di Pearson tra le probabilità non è calcolabile (servono almeno 2 punti).

8. Discussion

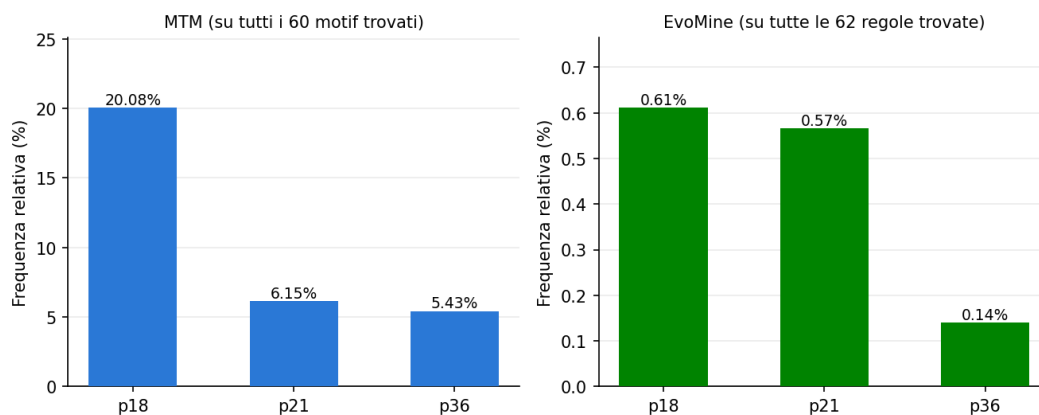
Nota (chiarimento della correlatrice): le 0 celle in comune tra le transition matrix finali (Sezione 7) sono un risultato atteso, non un difetto della pipeline — MTM ed EvoMine tracciano l'evoluzione della rete con meccanismi diversi per costruzione. La ‘forzatura’ di rappresentare entrambi come transition matrix con lo stesso formato di heatmap serve a ottenere una visualizzazione comune per un confronto qualitativo affiancato, non a garantire una sovrapposizione numerica: assi, codici e valori restano specifici di ciascun algoritmo. Il confronto realmente informativo tra le due fonti emerge invece dalla codifica canonica condivisa applicata a monte (Sezione 3) sui pattern individuali, prima della costruzione delle matrici.

Codice	Pattern	Ruolo MTM	Count MTM	Freq.rel. MTM	Ruolo EvoMine	Supp. EvoMine	Freq.rel. EvoMine
p18	0_1_0-0_2_1	prefisso 2 archi	643	20.08%	regola completa (1 body+1 head)	1570	0.61%

p21	0_1_0-2_0_1	prefisso 2 archi	197	6.15%	regola completa (1 body+1 head)	1453	0.57%
p36	0_1_0-1_0_1	prefisso 2 archi	174	5.43%	regola completa (1 body+1 head)	360	0.14%

Le frequenze relative sono calcolate sul totale dei pattern trovati da ciascun algoritmo — 3202 occorrenze cumulate sui 60 motif MTM (2 e 3 archi) al denominatore per MTM, 256498 unità di supporto cumulate sulle 62 regole EvoMine per EvoMine — l'analogo diretto per ciascuna fonte del proprio universo completo di pattern minati, non solo del sottoinsieme che compare come riga o colonna nella transition matrix finale (dati completi in `output_data/shared_patterns_comparison.csv`).

Frequenza relativa dei 3 pattern condivisi (codifica canonica GERANIO) — CollegeMsg, 1h



Le due fonti concordano sull'ordinamento qualitativo ($p18 > p21 > p36$ sia in MTM sia in EvoMine) ma differiscono di 30-40 volte in valore assoluto (es. p18: 20.08% in MTM contro 0.61% in EvoMine): un divario atteso, dato che i due algoritmi contano le occorrenze con meccanismi molto diversi — MTM conta motif distinti su tutta la sequenza di eventi, EvoMine conta il supporto su bucket temporali discretizzati (1h), tipicamente molto più numerosi. Il fatto che l'ordinamento relativo si conservi nonostante la scala assoluta sia incomparabile indica che i due algoritmi, pur misurando grandezze diverse, concordano su quali pattern a 2 archi siano strutturalmente più rilevanti nella rete CollegeMsg. MTM ed EvoMine risultano quindi complementari più che sovrapponibili, coerentemente con quanto già osservato nei Task 2-4: EvoMine esplora un numero maggiore di pattern a supporti e granularità diverse, mentre MTM fornisce probabilità condizionali normalizzate sul prefisso, utili a interpretare la sequenzialità dell'evoluzione della rete che EvoMine non cattura nello stesso modo.

9. Punto aperto — Clock sequenziale vs binario

Le due matrici hanno 0 celle in comune perché MTM usa timestamp sequenziali (t_0 , t_1 , t_2) mentre EvoMine usa un clock binario (body= t_0 , head= t_1). La stessa sequenza MTM “ $0 \rightarrow 1$ (t_0), $0 \rightarrow 2$ (t_1)” diventa in EvoMine “ $0 \rightarrow 1$ (t_0), $0 \rightarrow 2$ (t_0)” — due archi contemporanei invece di sequenziali. Le forme canoniche risultanti non coincidono mai, salvo nel caso già discusso allo Step 1 di preconditione a 1 solo arco. Soluzione proposta: trattare l'intero prefisso MTM come body a $t=0$ (anziché $t=0$, $t=1$, $t=2$ sequenziali), rendendo la convenzione di clock MTM strutturalmente identica a quella EvoMine. Da concordare con la correlatrice.

10. Conclusioni

- Forma canonica condivisa GERANIO implementata con successo su MTM ed EvoMine
- Top 23 regole EvoMine selezionate al gomito secondario della distribuzione del supporto
- Le due transition matrix non hanno celle in comune, a causa del problema del clock — punto aperto
- Le 0 celle in comune sono attese per costruzione (Sezione 8): MTM ed EvoMine tracciano l'evoluzione in modo diverso, la transition matrix condivisa serve al confronto qualitativo, non alla sovrapposizione numerica
- I 3 pattern condivisi dalla codifica canonica mostrano lo stesso ordinamento di frequenza relativa in entrambe le fonti ($p_{18} > p_{21} > p_{36}$), pur su scale assolute molto diverse — segno di complementarità tra MTM ed EvoMine
- Prossimo passo: decidere la convenzione di clock con la correlatrice e riallineare i timestamp MTM